

43. Un lutin, hors de la grille

Le fond est une grille de 8 en 8 ; un lutin ne l'est pas.

1. Ce qu'il faut comprendre

`sprite(numero, x, y, tuile)` pose un lutin **en pixels**, et non en cases : il peut se trouver à cheval entre deux tuiles du fond. C'est ce qui rend un mouvement doux possible.

Le premier argument est le **numéro du lutin**, de 0 à 39. C'est lui qu'on rappelle pour le déplacer, ou qu'on donne à `cacher(n)` pour l'ôter.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <sprite>` → [0.69.4. La fonction `sprite\(\)` — un lutin au pixel près](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **422 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	44 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code> — attend de pouvoir écrire une case
EcrireTexte	10 o	← <code>texte()</code> — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	112 o	← <code>CopierTuiles</code> — 6 lettres sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← <code>InstallerTransfert</code> — la petite routine qui copie les lutins
"IL FLOTTE"	9 o	← <code>texte()</code> — un texte gravé

Lettres gravées : EFILOT

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	× 1	23 o sur place, dans <code>main()</code>
<code>texte()</code>	× 1	11 o sur place, dans <code>main()</code> + <code>EcrireTexte</code> (10 o)
<code>image()</code>	× 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

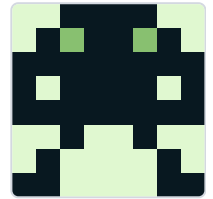
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : pose un personnage (un « lutin ») au milieu de l'écran.
//
// Un LUTIN (en anglais « sprite ») est un dessin de 8 × 8 qui n'est pas
// dans la grille des cases : il se pose AU PIXEL près, par-dessus le décor.
// L'écran fait 160 × 144 pixels.
// sprite(numéro, x, y, TUILE) : numéro = quel lutin (0, 1, 2 ...),
//                               x, y = sa place en pixels (coin en haut à gauche).
// Chez un lutin, les points (.) sont transparents : on voit le fond à travers.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <sprite>   // place un lutin de 8 × 8 au pixel près
#include <texte>    // écrit un texte à l'écran
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
  "..####..",
  ".#-##-#.",
  "#####",
  "#.####.#",
  "#####",
  "..#..#..",
  ".#...#.",
  "##...##",
};
```



à l'écran

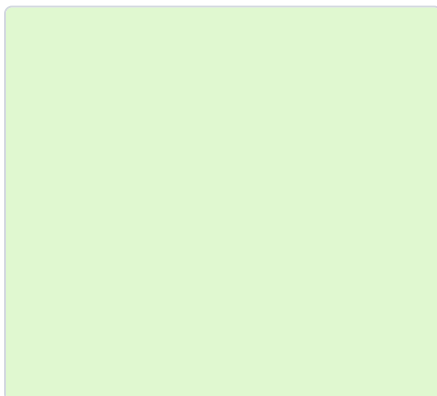
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite(0, 76, 68, HEROS); // Le jeu commence ici.
    sprite(0, 76, 68, HEROS); // Lutin 0, au pixel (76, 68) : à peu près au milieu.
    texte(3, 14, "IL FLOTTE"); // Un texte, dans la grille, ligne 14.

    while (true) {
        image(); // Répète pour toujours :
                // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un personnage au milieu, entre deux cases de la grille.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le lutin 0 est posé
- ✓ il est aux pixels demandés (76, 68)
- ✓ et donc PAS sur un multiple de huit
- ✓ le fond, lui, reste en cases

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom. étiquette. ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

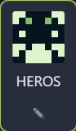
chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

HEROS

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuile n° 44 — poser(x, y, HEROS)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

HEROS ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

HEROS peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼